

Laporan Praktikum Kontrol Cerdas

SCIKIT-LEARN

Nama : Yanfirga Setyawan Putra Sinaga

NIM : 234308087

Kelas : TKA-6C

Akun Github (Tautan) : <https://github.com/yanfirgasinaga-spec>

Abstraact

Praktikum bertujuan untuk memahami implementasi machine learning menggunakan library Scikit-learn pada sistem klasifikasi berbasis visi komputer. Proses dilakukan melalui beberapa tahapan, yaitu pengumpulan dataset gambar menggunakan webcam, ekstraksi fitur landmark tangan dengan MediaPipe Hands, pelatihan model menggunakan algoritma Random Forest, serta pengujian sistem secara real-time. Data landmark yang diperoleh dinormalisasi dan disimpan untuk kemudian digunakan pada proses pelatihan dan evaluasi model. Hasil pengujian menunjukkan bahwa model mampu melakukan klasifikasi gesture tangan dengan tingkat akurasi yang baik. Selain itu, sistem juga dikembangkan untuk mendeteksi pose tubuh (duduk/berdiri) menggunakan MediaPipe Pose. Integrasi OpenCV, MediaPipe, dan Scikit-learn membuktikan bahwa pendekatan machine learning dapat diterapkan secara efektif dalam pengembangan sistem kontrol cerdas berbasis citra secara real-time.

Kata kunci: Scikit-learn, MediaPipe, Random Forest, OpenCV, Klasifikasi Gesture, Deteksi Pose

1. Pendahuluan

Scikit-learn merupakan salah satu library Python yang paling banyak digunakan dalam pengembangan machine learning, baik oleh pemula maupun praktisi data science. Library ini menyediakan berbagai algoritma siap pakai untuk keperluan klasifikasi, regresi, clustering, reduksi dimensi, hingga evaluasi model. Dengan struktur yang sederhana dan dokumentasi yang lengkap, Scikit-learn mempermudah pengguna dalam membangun model berbasis data tanpa harus merancang algoritmanya dari awal.

Dalam bidang visi komputer, machine learning memiliki peran penting dalam proses identifikasi objek, pengenalan pola, dan deteksi aktivitas. Pada praktikum ini, Scikit-learn digunakan untuk melatih model klasifikasi gesture tangan berdasarkan fitur landmark yang diekstraksi menggunakan MediaPipe Hands. Selain itu, MediaPipe Pose juga dimanfaatkan untuk mendeteksi pose tubuh sehingga sistem dapat digunakan sebagai kontrol cerdas yang bekerja secara real-time melalui integrasi dengan OpenCV.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami alur lengkap pengembangan model machine learning, mulai dari pengumpulan data, preprocessing,

ekstraksi fitur, pelatihan model, hingga implementasi langsung. Dengan demikian, praktikum ini memberikan pemahaman praktis mengenai penerapan machine learning dalam sistem berbasis visi komputer.

2. Tujuan

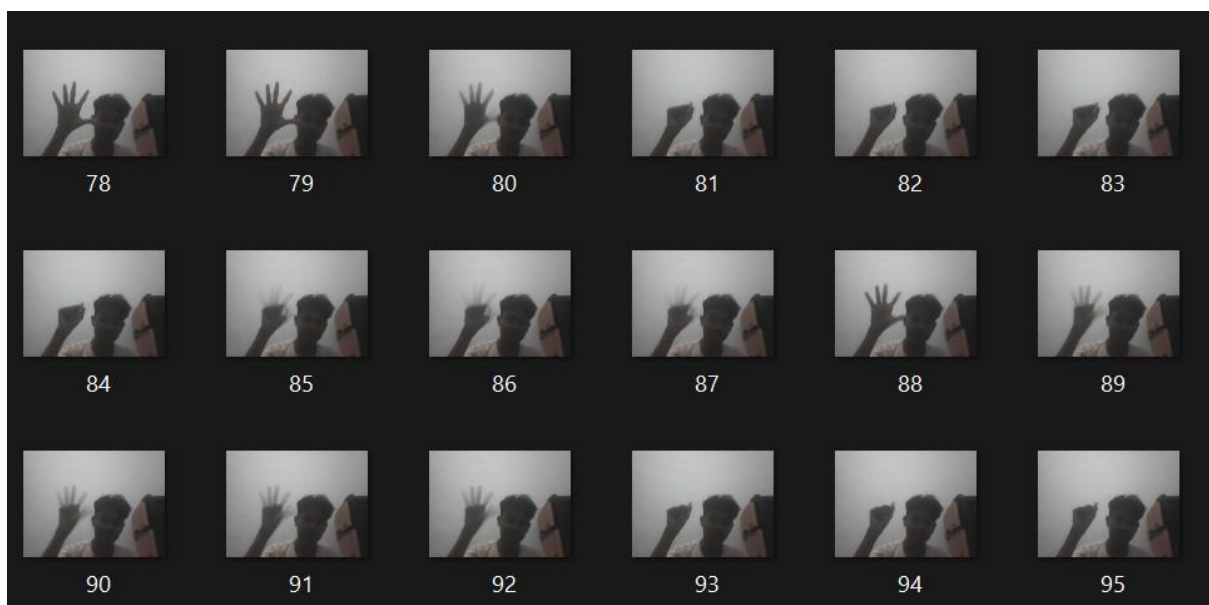
- Mempelajari dasar-dasar machine learning secara praktis melalui implementasi langsung pada data.
- Melatih kemampuan analisis data untuk menemukan pola atau hubungan dalam dataset.
- Memahami proses pembuatan model machine learning mulai dari preprocessing data, training model, hingga evaluasi hasil.

3. Manfaat

- Mempermudah proses belajar machine learning karena algoritma sudah tersedia dan mudah diimplementasikan.
- Mempercepat pembuatan program berbasis data tanpa harus membuat algoritma dari awal.
- Terintegrasi dengan Python yang merupakan bahasa pemrograman populer dan banyak digunakan di berbagai bidang teknologi.

4. Hasil Program

A. Percobaan A Pengumpulan data menggunakan webcam



```

D: > Semester 6 > Kontrol Cerdas > Praktikum 3 > import os A.py > ...
1  import os
2  from time import sleep
3  import cv2
4  DATA_DIR = 'D:\Semester 6\Kontrol Cerdas\Praktikum 3\DATA2'
5  if not os.path.exists(DATA_DIR) :
6      os.makedirs(DATA_DIR)
7  number_of_classes = 2
8  dataset_size = 100
9  cap = cv2.VideoCapture(0)
10
11  for j in range(number_of_classes):
12      if not os.path.exists(os.path.join(DATA_DIR, str(j))):
13          os.makedirs(os.path.join(DATA_DIR, str(j)))
14          print('Collecting data for class{}'.format(j))
15
16          while True:
17              ret, frame = cap.read()
18              cv2.putText(frame, 'Redi? Pencet "Q"! ', (100, 50), cv2.FONT_ITALIC, 1.3, (0, 225, 0), 3)
19              cv2.imshow('frame', frame)
20              if cv2.waitKey(15) == ord('q'):
21                  break
22
23              counter = 0
24              while counter < dataset_size:
25                  ret, frame = cap.read()
26                  cv2.imshow('frame', frame)
27                  cv2.waitKey(10)
28                  cv2.imwrite(os.path.join(DATA_DIR, str(j), '{}.jpg'.format(counter)), frame)
29                  counter += 1
30              cap.release()
31              cv2.destroyAllWindows()

```

B. Percobaan B Ekstraksi data dari dataset

```

D: > Semester 6 > Kontrol Cerdas > Praktikum 3 > import os B.py > ...
1  import os
2  from time import sleep
3  import cv2
4  DATA_DIR = 'D:\Semester 6\Kontrol Cerdas\Praktikum 3\DATA2'
5  if not os.path.exists(DATA_DIR) :
6      os.makedirs(DATA_DIR)
7  number_of_classes = 2
8  dataset_size = 100
9  cap = cv2.VideoCapture(0)
10
11  for j in range(number_of_classes):
12      if not os.path.exists(os.path.join(DATA_DIR, str(j))):
13          os.makedirs(os.path.join(DATA_DIR, str(j)))
14          print('collecting data for class{}'.format(j))
15
16          while True:
17              ret, frame = cap.read()
18              cv2.putText(frame, 'Redi? Pencet "Q"! ', (100, 50), cv2.FONT_ITALIC, 1.3, (0, 225, 0), 3)
19              cv2.imshow('frame', frame)
20              if cv2.waitKey(15) == ord('q'):
21                  break
22
23              counter = 0
24              while counter < dataset_size:
25                  ret, frame = cap.read()
26                  cv2.imshow('frame', frame)
27                  cv2.waitKey(10)
28                  cv2.imwrite(os.path.join(DATA_DIR, str(j), '{}.jpg'.format(counter)), frame)
29                  counter += 1
30              cap.release()
31              cv2.destroyAllWindows()

```

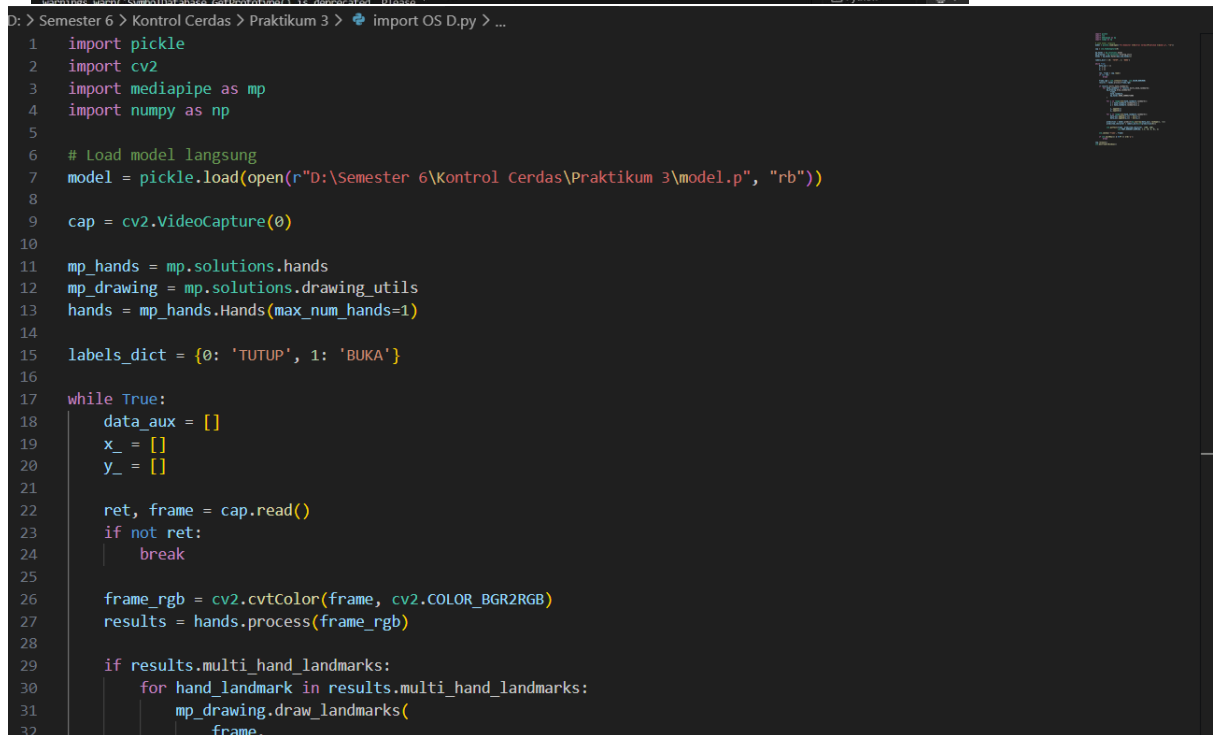
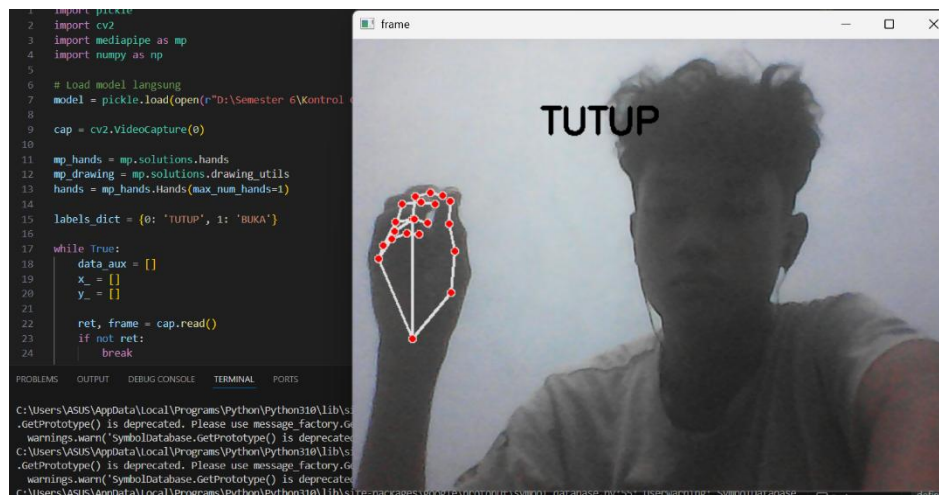
C. Percobaan C Pelatihan dan Pengujian

```

D: > Semester 6 > Kontrol Cerdas > Praktikum 3 > import os C.py > ...
1  import pickle
2  import numpy as np
3  from sklearn.model_selection import train_test_split
4  from sklearn.ensemble import RandomForestClassifier
5  from sklearn.metrics import accuracy_score
6
7  data_dict = pickle.load(open(r"D:\Semester 6\Kontrol Cerdas\Praktikum 3\DATA.pickle", "rb"))
8  clean_data = []
9  clean_labels = []
10
11  for d, lbl in zip(data_dict['data'], data_dict['labels']):
12      arr = np.array(d)
13      if arr.shape == (42,):
14          clean_data.append(arr)
15          clean_labels.append(lbl)
16  clean_data = np.array(clean_data)
17  clean_labels = np.array(clean_labels)
18  print("Data bersih:", clean_data.shape)
19  print("Label bersih:", clean_labels.shape)
20
21
22  x_train, x_test, y_train, y_test = train_test_split(
23      clean_data, clean_labels, test_size=0.2, shuffle=True, stratify=clean_labels
24  )
25
26  model = RandomForestClassifier()
27  model.fit(x_train, y_train)
28
29  y_pred = model.predict(x_test)
30  score = accuracy_score(y_pred, y_test)
31  print(f"{score * 100:.2f}% of samples were classified correctly!")
32

```

D. Percobaan D

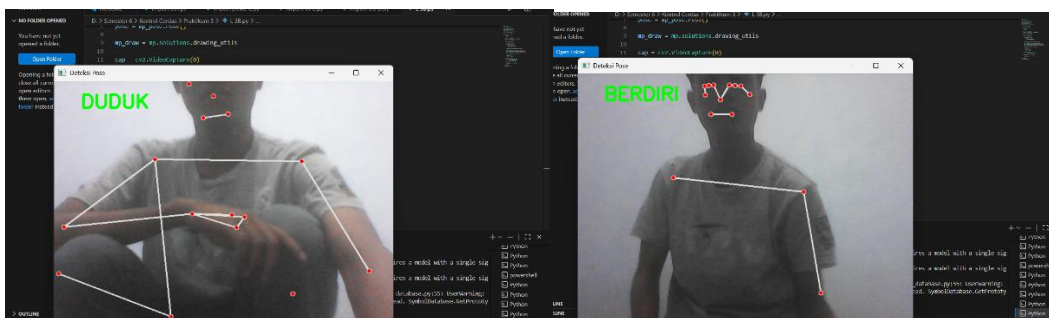


```

D: > Semester 6 > Kontrol Cerdas > Praktikum 3 > import OS D.py > ...
32     frame,
33     hand_landmark,
34     mp_hands.HAND_CONNECTIONS
35 )
36
37 for i in range(len(hand_landmark.landmark)):
38     x = hand_landmark.landmark[i].x
39     y = hand_landmark.landmark[i].y
40
41     x_.append(x)
42     y_.append(y)
43
44 for i in range(len(hand_landmark.landmark)):
45     data_aux.append(x_[i] - min(x_))
46     data_aux.append(y_[i] - min(y_))
47
48 prediction = model.predict(np.asarray(data_aux).reshape(1, -1))
49 predicted_character = labels_dict[int(prediction[0])]
50
51 cv2.putText(frame, predicted_character, (200, 100),
52             cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 0, 0), 3)
53
54 cv2.imshow('frame', frame)
55
56 if cv2.waitKey(1) & 0xFF == ord('q'):
57     break
58
59 cap.release()
60 cv2.destroyAllWindows()

```

E. Latihan B



```

D: > Semester 6 > Kontrol Cerdas > Praktikum 3 > L 3B.py > ...
1  import cv2
2  import mediapipe as mp
3  import numpy as np
4
5  # Inisialisasi mediapipe pose
6  mp_pose = mp.solutions.pose
7  pose = mp_pose.Pose()
8
9  mp_draw = mp.solutions.drawing_utils
10
11 cap = cv2.VideoCapture(0)
12
13 while True:
14     ret, frame = cap.read()
15     if not ret:
16         break
17
18     frame = cv2.flip(frame, 1)
19
20     rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
21     results = pose.process(rgb)
22
23     text = "Tidak terdeteksi"
24
25     if results.pose_landmarks:
26         mp_draw.draw_landmarks(
27             frame,
28             results.pose_landmarks,
29             mp_pose.POSE_CONNECTIONS
30         )
31
32

```

```

32
33     landmarks = results.pose_landmarks.landmark
34
35     # Titik penting
36     hip = landmarks[mp_pose.PoseLandmark.LEFT_HIP.value]
37     knee = landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value]
38     ankle = landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value]
39
40     # Perbandingan posisi vertikal (y)
41     # Semakin kecil nilai y = semakin atas
42
43     if knee.y > hip.y:
44         text = "BERDIRI"
45     else:
46         text = "DUDUK"
47
48     cv2.putText(frame, text, (50, 50),
49                 cv2.FONT_HERSHEY_SIMPLEX,
50                 1.2, (0, 255, 0), 3)
51
52     cv2.imshow("Deteksi Pose", frame)
53
54     if cv2.waitKey(1) & 0xFF == ord('q'):
55         break
56
57 cap.release()
58 cv2.destroyAllWindows()

```

5. Analisis Program

Pada percobaan A dilakukan proses perekaman gesture tangan menggunakan webcam. Hasil pengamatan menunjukkan bahwa kualitas data sangat bergantung pada pencahayaan, posisi tangan, dan konsistensi gesture. Kamera mampu menangkap gambar dengan baik, namun beberapa frame perlu diulang karena tangan tidak terdeteksi secara optimal. Percobaan ini berhasil menghasilkan dataset awal yang cukup untuk tahap ekstraksi fitur.

Pada percobaan B, ambar-gambar hasil pengambilan data diproses menggunakan MediaPipe Hands untuk memperoleh 21 titik landmark tangan. Pada proses ini, sebagian gambar tidak dapat diproses karena tangan tidak terlihat jelas atau keluar dari frame. Data landmark yang berhasil diekstraksi disimpan sebagai array numerik. Percobaan ini membuktikan bahwa MediaPipe cukup efektif, namun sensitif terhadap posisi dan pencahayaan.

Pada percobaan C, dataset landmark dibagi menjadi data pelatihan dan pengujian menggunakan *train-test split*. Model yang digunakan adalah Random Forest Classifier. Hasil pengujian menunjukkan bahwa model mampu mengenali gesture dengan tingkat akurasi yang baik. Hal ini menunjukkan bahwa fitur landmark yang diekstraksi sudah cukup representatif. Random Forest juga terbukti stabil meskipun dataset tidak terlalu besar.

Percobaan D melakukan pengujian model secara real-time menggunakan OpenCV. Kamera menangkap frame, lalu MediaPipe mendeteksi landmark dan model memberikan prediksi gesture. Implementasi berjalan baik, sistem dapat mengenali gesture dengan cepat dan responsif. Beberapa kendala terjadi ketika tangan terlalu jauh dari kamera atau pencahayaan berubah secara drastis. Namun secara umum, percobaan ini berhasil menunjukkan bagaimana model machine learning dapat diintegrasikan dalam aplikasi real-time.

Pada latihan B, MediaPipe Pose digunakan untuk mendeteksi postur tubuh dan menentukan apakah seseorang sedang duduk atau berdiri. Proses dimulai dengan

menangkap gambar dari kamera, kemudian MediaPipe mengidentifikasi titik-titik penting tubuh seperti bahu, pinggul, dan lutut.

Analisis menunjukkan bahwa perbandingan posisi landmark tertentu (misalnya jarak vertikal antara pinggul dan lutut) dapat digunakan untuk membedakan dua kondisi tersebut. Sistem dapat mendeteksi pose dasar ini dengan cukup akurat.

Namun ada beberapa hal yang mempengaruhi performa kamera harus menangkap tubuh bagian atas dengan jelas, pose tubuh harus dalam posisi tegak lurus terhadap kamera, dan jika pengguna terlalu membungkuk atau kamera terlalu rendah, deteksi bisa salah.

Latihan ini menunjukkan bahwa MediaPipe tidak hanya efektif untuk tangan, tetapi juga sangat baik dalam mendeteksi pose tubuh untuk aplikasi kontrol cerdas sederhana.

6. Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa:

1. Pengumpulan data menggunakan webcam berhasil memberikan dataset gesture yang cukup baik, meskipun sangat dipengaruhi oleh kualitas pencahayaan dan posisi tangan.
2. MediaPipe terbukti efektif dalam mengekstraksi landmark tangan maupun landmark tubuh untuk keperluan machine learning.
3. Algoritma Random Forest memberikan akurasi tinggi dan mudah diimplementasikan untuk klasifikasi gesture.
4. Integrasi OpenCV, MediaPipe, dan Scikit-Learn berhasil menciptakan sistem deteksi gesture dan pose tubuh secara real-time.
5. Model dapat dikembangkan lebih lanjut dengan menambah dataset, memperbaiki kondisi pengambilan gambar, dan menambah variasi gesture atau pose.

7. Saran

- Jumlah dataset sebaiknya ditambah agar model memiliki variasi data yang lebih banyak dan meningkatkan akurasi.
- Kondisi pencahayaan dan posisi tangan saat pengambilan data perlu diperhatikan agar hasil deteksi lebih stabil.

8. Referensi

- Belajar Python– Situs Open Source Tutorial Pemrograman Python Ba hasa Indonesia <https://belajarpython.com>